

DAY 12

Web3 Frontend Development with ethers.js v6

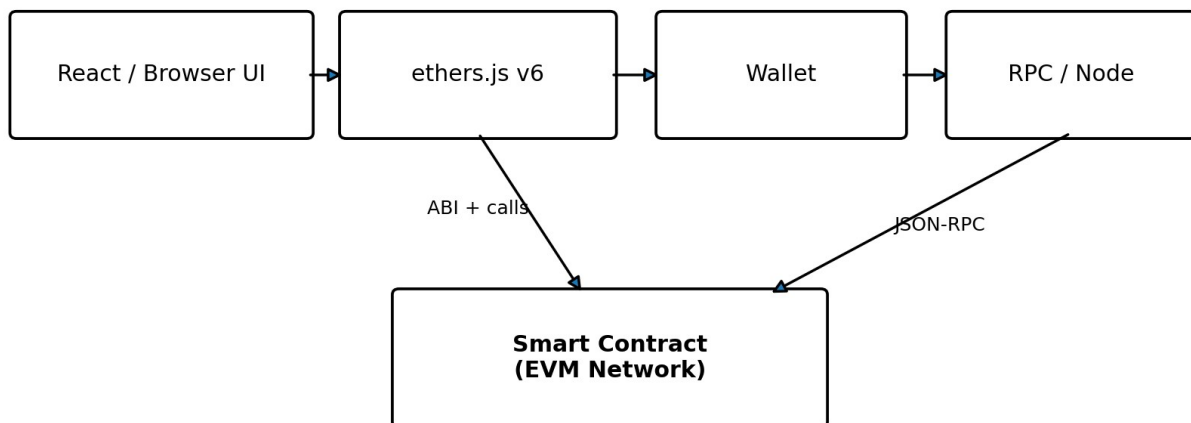
Complete 60-Minute Beginner Lecture

Provider + Signer + ABI + Contract Calls

Wallet Connection + Network Checks + Transaction UX

Events + Errors + Security + Client Delivery

Web3 Frontend Architecture



Career outcome

By the end of this lesson, you should be able to connect a browser wallet, read blockchain state, submit a state-changing transaction, wait for confirmation, display events/errors, and explain the frontend architecture to a client.

Target: Beginner -> Job-ready Web3 frontend foundation

1. Learning Objectives & 60-Minute Plan

Day 12 changes your perspective from "I can write a smart contract" to "I can build a usable application around a smart contract." The browser does not execute Solidity; it uses wallet/provider APIs and an ABI to communicate with an EVM network.

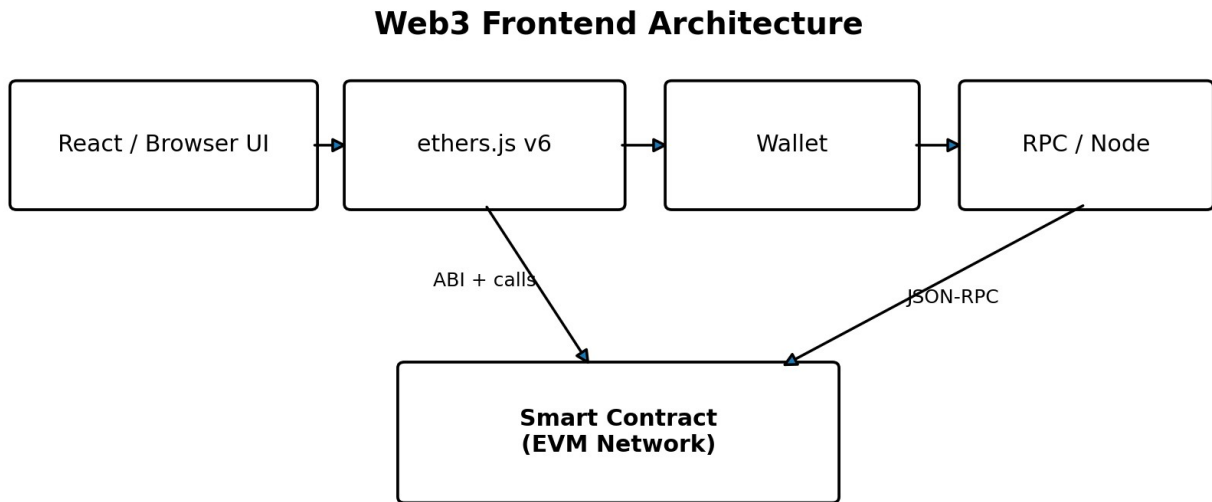
Minute	Topic	Outcome
0-5	Web3 frontend mental model	Understand browser -> ethers -> wallet/RPC -> contract
5-12	Provider, signer and runner	Know which object reads and which object authorizes writes
12-20	Wallet connection + network	Connect accounts and validate chain safely
20-30	ABI + contract reads	Create Contract objects and display state
30-40	Writes + transaction lifecycle	Submit, wait, confirm and refresh UI
40-47	Events + reactive UI	Listen for logs and re-read authoritative state
47-53	Errors + security	Handle rejection, revert, wrong chain and unsafe patterns
53-58	Hands-on React lab	Build a small contract dashboard
58-60	Freelancer checklist + review	Translate skill into client delivery

- Explain the difference between Provider and Signer.
- Connect to an EIP-1193 wallet provider from ethers.js v6.
- Create a Contract object from address + ABI + runner.
- Perform read-only calls and state-changing transactions.
- Track pending/confirmed/error states in the UI.
- React to account/chain changes and contract events.
- Apply frontend security practices appropriate for client projects.

Teacher checkpoint

Before coding, ask students: "Which part of a dApp holds the private key?" Correct answer: the wallet, not your React app and not ethers.js.

2. The Web3 Frontend Mental Model



A Web3 frontend is still a normal web application. The difference is that part of its data and business logic lives on a blockchain. The frontend must communicate with that blockchain through a provider and, when a user authorizes a write, through a signer controlled by a wallet.

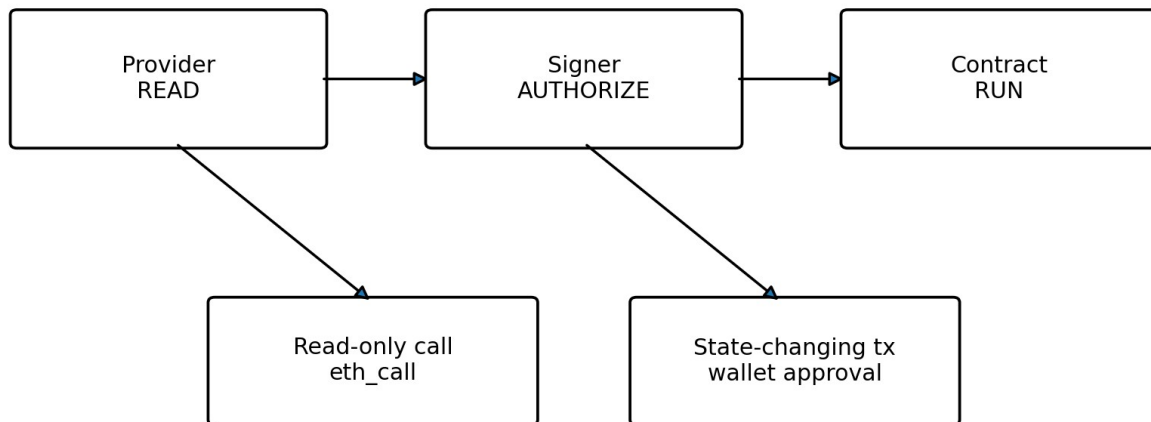
Layer	Responsibility	Typical technology
UI	Buttons, forms, loading states, validation	React / Next.js / browser JS
Web3 library	Encode calls, decode results, normalize provider APIs	ethers.js v6
Wallet	Account access, signatures, transaction approval	MetaMask or compatible wallet
Provider/RPC	Read chain state, broadcast signed transactions	Injected EIP-1193 provider or JSON-RPC endpoint
Contract	Business rules and persistent EVM state	Solidity bytecode + storage

Freelancer language

When a client says "connect my website to my smart contract," your job is usually: wallet connection + correct ABI/address + read calls + write calls + transaction UX + chain checks + error handling.

3. Provider vs Signer - The Most Important Distinction

Provider vs Signer - the key frontend distinction



Provider

A Provider is your read-oriented connection to Ethereum-compatible blockchain data. It can query balances, blocks, receipts, logs and simulate/read smart-contract functions. A provider does not magically own the user's private key.

```
const provider = new ethers.BrowserProvider(window.ethereum);

const blockNumber = await provider.getBlockNumber();
const balance = await provider.getBalance("0x...");
console.log(ethers.formatEther(balance));
```

Signer

A Signer represents an account that can authorize actions. In a browser wallet flow, the wallet manages the private key and ethers obtains a signer interface after account permission is available.

```
await provider.send("eth_requestAccounts", []);
const signer = await provider.getSigner();
const userAddress = await signer.getAddress();
```

Rule

Provider = read/observe. Signer = authorize/sign. A contract connected only to a Provider is read-only; a contract connected to a Signer can submit state-changing transactions if the account has permission.

4. Browser Wallet APIs: EIP-1193 and Multi-Wallet Discovery

Browser wallets expose a provider API to web applications. EIP-1193 standardizes a minimal event-driven Ethereum Provider interface, including a request method and events such as account and chain changes. ethers.js BrowserProvider wraps an EIP-1193-compatible provider so your app can use the ethers API.

```
if (!window.ethereum) {
  throw new Error("No injected wallet provider detected");
}

const provider = new ethers.BrowserProvider(window.ethereum);
await provider.send("eth_requestAccounts", []);
```

Concept	Why it matters
request(...)	Wallet/provider entry point for JSON-RPC requests
accountsChanged	UI must update when the selected account changes
chainChanged	UI must verify/reload state for the new network
disconnect	App should surface disconnected/unavailable provider state
EIP-6963	Modern discovery mechanism for environments with multiple injected wallet providers

Production note

For a beginner lab, window.ethereum is convenient. For a polished multi-wallet application, do not assume there is only one injected provider; EIP-6963 exists specifically to improve discovery when multiple wallets are installed.

5. Connect Wallet, Account and Network Safely

Connection flow

1. Detect a wallet/provider.
2. Request account access only after a user action such as clicking Connect Wallet.
3. Create BrowserProvider.
4. Get the Signer and user address.
5. Read the active network and compare its chain ID with your deployment configuration.
6. Only enable contract-write controls when the wallet and network are ready.

```
const TARGET_CHAIN_ID = 11155111n; // example testnet config

const provider = new ethers.BrowserProvider(window.ethereum);
await provider.send("eth_requestAccounts", []);

const signer = await provider.getSigner();
const address = await signer.getAddress();
const network = await provider.getNetwork();

if (network.chainId !== TARGET_CHAIN_ID) {
  throw new Error(`Wrong network: ${network.chainId}`);
}

console.log({ address, chainId: network.chainId.toString() });
```

Important ethers v6 detail

Many numeric blockchain values are represented as JavaScript bigint values in ethers v6. Avoid mixing bigint with ordinary number arithmetic without conversion.

UI state	Recommended behavior
No wallet	Show install/connect guidance; keep write buttons disabled
Not connected	Show Connect Wallet
Wrong chain	Show expected network and a switch-network action if supported
Ready	Show account, balances, contract state and enabled actions

6. ABI: The Contract Interface Your Frontend Understands

The ABI describes callable functions, events and data types. It is the bridge between human-friendly JavaScript calls and EVM calldata/log encoding. A frontend usually needs the deployment address and ABI for the exact contract/interface it intends to call.

```
export const CONTRACT_ADDRESS = "0xYourDeployedContract";

export const ABI = [
  "function projectName() view returns (string)",
  "function owner() view returns (address)",
  "function setProjectName(string newName)",
  "event ProjectNameChanged(address indexed editor, string oldName, string newName)"
];

const readContract = new ethers.Contract(
  CONTRACT_ADDRESS,
  ABI,
  provider
);

const writeContract = new ethers.Contract(
  CONTRACT_ADDRESS,
  ABI,
  signer
);
```

Client debugging clue

If a frontend says "function is not a function" or calls the wrong selector, verify the ABI, contract address, network, proxy address vs implementation address, and deployed contract version before blaming the wallet.

7. Read-Only Contract Calls

Read-only calls do not modify blockchain state and normally do not ask the user to sign a transaction. They are ideal for dashboards: balances, owners, token metadata, staking positions, order status and configuration values.

```
async function loadProject(contract, provider, user) {
  const [name, owner, contractBalance] = await Promise.all([
    contract.projectName(),
    contract.owner(),
    provider.getBalance(await contract.getAddress())
  ]);

  return {
    name,
    owner,
    contractBalanceEth: ethers.formatEther(contractBalance),
    isOwner: owner.toLowerCase() === user.toLowerCase()
  };
}
```

Read pattern	Example
Native balance	provider.getBalance(address)
Public/view function	contract.owner()
Token decimals/name/symbol	erc20.decimals(), name(), symbol()
Mapping getter	contract.positions(user)
Past logs	provider.getLogs(filter) or contract query helpers

Performance habit

Avoid serially awaiting five independent reads when they can run concurrently. Promise.all can reduce visible UI latency, but do not overload RPC endpoints with unbounded requests.

8. State-Changing Contract Calls

A state-changing function creates a transaction request. The user must approve/sign it in the wallet. Submitting the transaction does not mean it has already been mined or finalized. Your UI must represent the transaction lifecycle.

```
async function updateProjectName(contract, newName) {
  if (!newName.trim()) throw new Error("Project name is required");

  const tx = await contract.setProjectName(newName.trim());
  console.log("submitted:", tx.hash);

  const receipt = await tx.wait();
  console.log("confirmed in block:", receipt.blockNumber);

  return receipt;
}
```

Moment	What the user sees	Developer meaning
Before wallet prompt	Awaiting approval	Transaction request prepared
After signature/broadcast	Submitted + tx hash	Network has transaction
Before inclusion	Pending	Not yet confirmed
Receipt success	Confirmed	State transition included
Revert/rejection	Failed/cancelled	Explain what happened; do not show fake success

Never do this

Do not display "Payment successful" immediately after `contract.someWrite()`. The returned transaction response only proves submission; wait for a receipt/confirmation policy appropriate to the application.

9. Transaction UX State Machine

Transaction UX State Machine



Also handle: user rejection | wrong network | revert | replacement | dropped transaction

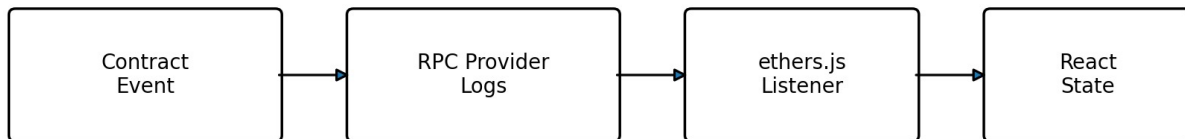
```
setTxState({ status: "awaiting-signature" });
try {
  const tx = await contract.setProjectName(name);
  setTxState({ status: "pending", hash: tx.hash });

  const receipt = await tx.wait();
  setTxState({ status: "confirmed", hash: tx.hash, receipt });
} catch (err) {
  setTxState({ status: "error", message: humanizeWeb3Error(err) });
}
```

- Disable duplicate-submit buttons while a write is pending.
- Show the transaction hash so the user can inspect it in a block explorer.
- Separate user rejection from contract revert and RPC/network failure.
- After confirmation, re-read contract state instead of assuming local UI arithmetic is correct.
- For high-value applications, define a confirmation/finality policy instead of treating the first receipt as universal finality.

10. Events and Reactive Interfaces

Event-driven UI refresh



Use listeners for live UX, but re-read authoritative contract state when correctness matters.

Solidity events become EVM logs that off-chain software can index and observe. A frontend can listen for new events to refresh the interface, notify the user, or invalidate cached data.

```
function subscribeToProjectChanges(contract, onChange) {
  const listener = (editor, oldName, newName) => {
    onChange({ editor, oldName, newName });
  };

  contract.on("ProjectNameChanged", listener);

  return () => {
    contract.off("ProjectNameChanged", listener);
  };
}
```

React cleanup

Always remove listeners when components unmount or dependencies change. Otherwise re-renders can create duplicate subscriptions and repeated notifications.

Events are excellent for responsiveness, but when correctness matters, treat contract state as authoritative. An event tells you something happened; a follow-up read tells you the state your UI should display now.

11. Account and Chain Change Handling

A connected dApp is not static. Users can change accounts, switch networks, lock their wallet or remove permissions. Your app must react instead of continuing to display stale authorization state.

```
function installWalletListeners() {
  if (!window.ethereum) return () => {};

  const handleAccounts = (accounts) => {
    console.log("accounts changed", accounts);
    // Rebuild signer/contract state or mark disconnected.
  };

  const handleChain = (chainId) => {
    console.log("chain changed", chainId);
    // Re-read network-specific deployment config.
  };

  window.ethereum.on("accountsChanged", handleAccounts);
  window.ethereum.on("chainChanged", handleChain);

  return () => {
    window.ethereum.removeListener("accountsChanged", handleAccounts);
    window.ethereum.removeListener("chainChanged", handleChain);
  };
}
```

Change	What can become stale
Account change	Signer, address, role permissions, token balances, allowances
Chain change	Contract address, ABI/version assumptions, explorer links, currency symbol
Disconnect	All wallet-authorized writes and account-specific data

12. Error Handling That Clients Notice

Good Web3 applications turn low-level failures into useful user messages without hiding technical details needed for debugging. Do not use a single "Transaction failed" message for every situation.

Failure class	Example user message	Developer action
User rejected	Transaction was cancelled in your wallet.	Return to ready state; do not retry automatically.
Wrong network	Please switch to the supported network.	Compare chain ID with deployment config.
Contract revert	Contract rejected this action: insufficient permission.	Surface reason/custom-error data when safely available.
Insufficient funds	Not enough native token for value + gas.	Show required asset/network.
RPC/network	Unable to reach blockchain provider.	Retry reads with backoff or alternate provider policy.

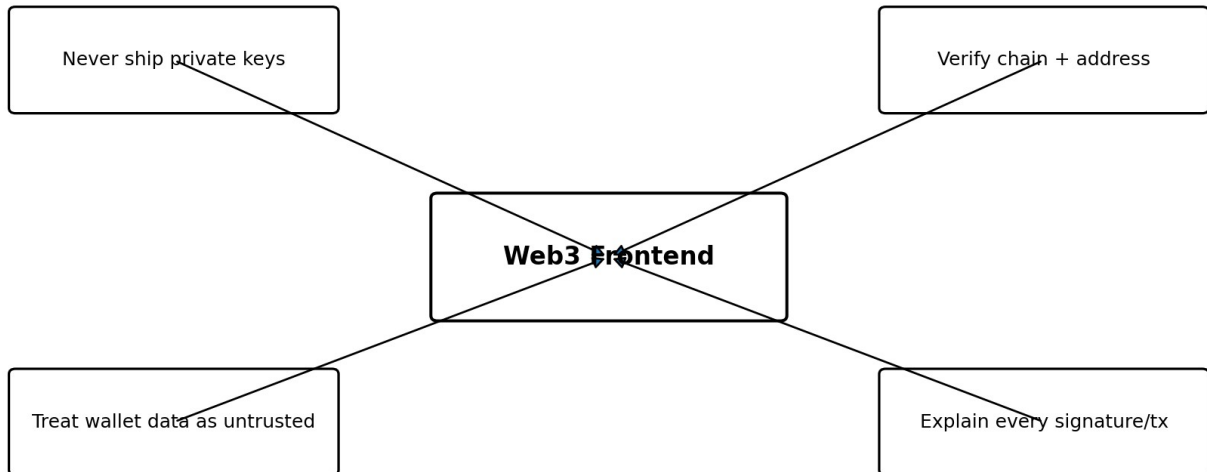
```
function humanizeWeb3Error(err) {
  if (err?.code === "ACTION_REJECTED" || err?.code === 4001) {
    return "You cancelled the wallet request.";
  }
  if (err?.shortMessage) return err.shortMessage;
  if (err?.reason) return err.reason;
  return "Blockchain request failed. Check network and try again.";
}
```

Security/UX balance

Do not dump enormous raw error objects or sensitive app configuration into end-user alerts. Log diagnostic context appropriately, and present a concise action-oriented message to the user.

13. Web3 Frontend Security Checklist

Frontend security is part of smart-contract security



- Never put deployer/private keys or wallet seed phrases in browser code, .env variables that are bundled to the client, Git repositories or analytics logs.
- Assume browser input, query parameters, wallet-provided account data and remote API data are untrusted until validated.
- Verify expected chain ID and contract address before enabling state-changing actions.
- Show the user exactly what an action will do before requesting a signature or transaction.
- Do not ask users to sign opaque messages you cannot explain.
- Use checksummed/validated addresses when accepting address inputs, and display enough of an address to avoid misleading truncation.
- Treat frontend access controls as UX only; real authorization must be enforced by the smart contract.
- Do not rely on hidden buttons to protect privileged functions.

Critical principle

A secure React condition such as `if (isAdmin)` is not an on-chain security boundary. Attackers can call the contract directly. The Solidity contract must enforce authorization.

14. Hands-on Contract: FreelancerBoard

For the frontend lab, use a small contract with one read flow, one privileged write flow and one event. Deploy it only to a local chain or test network for practice.

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.24;

contract FreelancerBoard {
    address public immutable owner;
    string public projectName;

    error NotOwner();
    error EmptyName();

    event ProjectNameChanged(
        address indexed editor,
        string oldName,
        string newName
    );

    constructor(string memory initialName) {
        if (bytes(initialName).length == 0) revert EmptyName();
        owner = msg.sender;
        projectName = initialName;
    }

    function setProjectName(string calldata newName) external {
        if (msg.sender != owner) revert NotOwner();
        if (bytes(newName).length == 0) revert EmptyName();

        string memory oldName = projectName;
        projectName = newName;
        emit ProjectNameChanged(msg.sender, oldName, newName);
    }
}
```

Learning design

This is intentionally simple. Day 12 is about frontend integration, not adding more Solidity complexity. The same frontend pattern later applies to ERC-20, NFT, staking, escrow and DeFi contracts.

15. React + Vite + ethers.js v6 Setup

```
npm create vite@latest day12-web3 -- --template react
cd day12-web3
npm install
npm install ethers
npm run dev
```

Suggested project structure

```
day12-web3/
├─ src/
│  ├─ App.jsx
│  ├─ contract.js
│  ├─ abi/
│  │  └─ FreelancerBoard.json
│  └─ main.jsx
├─ .env.example
├─ package.json
└─ README.md

// src/contract.js
export const CONTRACT_ADDRESS = "0x...";
export const EXPECTED_CHAIN_ID = 11155111n;

export const ABI = [
  "function owner() view returns (address)",
  "function projectName() view returns (string)",
  "function setProjectName(string newName)",
  "event ProjectNameChanged(address indexed editor, string oldName, string newName)"
];
```

Deployment configuration

Keep chain-specific contract addresses in explicit configuration. A frequent freelance bug is using the right ABI against the wrong address or the right address on the wrong network.

16. React Lab - Connection and Reads

```
import { useState } from "react";
import { ethers } from "ethers";
import { ABI, CONTRACT_ADDRESS, EXPECTED_CHAIN_ID } from "./contract";

export default function App() {
  const [account, setAccount] = useState("");
  const [projectName, setProjectName] = useState("");
  const [owner, setOwner] = useState("");
  const [status, setStatus] = useState("Disconnected");

  async function connect() {
    if (!window.ethereum) throw new Error("Wallet not found");

    const provider = new ethers.BrowserProvider(window.ethereum);
    await provider.send("eth_requestAccounts", []);

    const signer = await provider.getSigner();
    const address = await signer.getAddress();
    const network = await provider.getNetwork();

    if (network.chainId !== EXPECTED_CHAIN_ID) {
      throw new Error("Wrong network");
    }

    const contract = new ethers.Contract(CONTRACT_ADDRESS, ABI, provider);
    const [name, contractOwner] = await Promise.all([
      contract.projectName(),
      contract.owner()
    ]);

    setAccount(address);
    setProjectName(name);
    setOwner(contractOwner);
    setStatus("Ready");
  }
}
```

At this stage the Contract is connected to the Provider because the code is only reading state. The Signer will be used for the write function.

17. React Lab - Write, Wait and Refresh

```

async function saveName(newName) {
  const provider = new ethers.BrowserProvider(window.ethereum);
  const signer = await provider.getSigner();
  const contract = new ethers.Contract(CONTRACT_ADDRESS, ABI, signer);

  setStatus("Awaiting wallet approval");
  try {
    const tx = await contract.setProjectName(newName);
    setStatus(`Pending: ${tx.hash}`);

    await tx.wait();

    const readContract = contract.connect(provider);
    const freshName = await readContract.projectName();
    setProjectName(freshName);
    setStatus("Confirmed");
  } catch (err) {
    setStatus(humanizeWeb3Error(err));
  }
}

return (
  <main>
    <button onClick={connect}>Connect Wallet</button>
    <p>Account: {account || "Not connected"}</p>
    <p>Project: {projectName || "-"}</p>
    <p>Owner: {owner || "-"}</p>
    <p>Status: {status}</p>
  </main>
);
}

```

Extension exercise

Add an input and Save button, but enable it only when `account.toLowerCase() === owner.toLowerCase()`. Remember: this is UX convenience only; the contract still enforces owner authorization.

18. React Lab - Event Subscription Pattern

```
import { useEffect } from "react";

useEffect(() => {
  if (!window.ethereum) return;

  const provider = new ethers.BrowserProvider(window.ethereum);
  const contract = new ethers.Contract(CONTRACT_ADDRESS, ABI, provider);

  const onChange = async () => {
    const freshName = await contract.projectName();
    setProjectName(freshName);
  };

  contract.on("ProjectNameChanged", onChange);

  return () => {
    contract.off("ProjectNameChanged", onChange);
  };
}, []);
```

- Use event listeners to make the UI responsive to transactions submitted elsewhere.
- Clean up listeners to prevent duplicate subscriptions.
- If the app can switch chains, rebuild the provider/contract subscription for the selected deployment.
- For large production apps, consider a data/indexing layer rather than attaching many raw RPC listeners from every client browser.

19. Debugging Matrix for Upwork/Client Jobs

Symptom	Most likely checks
Connect button does nothing	Is provider injected? Is request called from user gesture? Browser console error?
Wrong balance / empty reads	Correct chain? Correct address? Correct decimals? Correct RPC?
Function missing	ABI mismatch? Proxy vs implementation? Wrong contract version?
Transaction reverts	Caller role? value/allowance? contract preconditions? simulation/revert data?
Transaction stays pending	Gas/network congestion, nonce/replacement, wallet/provider status
UI says success but state unchanged	Did code await receipt? Was transaction on correct chain? Was it reverted/replaced?
Events fire multiple times	Duplicate listeners? Missing React cleanup? Multiple providers?
Admin button hidden but function exploitable	Frontend is not authorization; inspect Solidity access control

Client workflow

Collect exact evidence: website URL/version, wallet, network, account, contract address, transaction hash, function, expected result, actual result, console/RPC error. Reproduce on a test environment before changing code.

20. Freelancer Scenarios - Translate Client Language

Client request	Technical interpretation	Deliverable
"Connect MetaMask to my site"	Wallet provider detection + account permission + account/chain state	Connect flow + state handling + network guard
"Show my token balance"	ERC-20 ABI + balanceOf + decimals + formatting	Readable token balance component
"Make the Buy button work"	Signer-connected contract call + value/arguments + tx lifecycle	Validated write action + pending/confirmed/error UX
"Dashboard does not refresh"	Event subscription or post-confirmation state refresh	Deterministic refresh strategy
"Works locally but not production"	Environment config, chain/address/RPC mismatch, CSP/network issues	Deployment checklist + reproducible config

Sample proposal language

I will first verify the deployed contract address, ABI and target chain. Then I will connect the frontend through ethers.js, implement wallet/account/network handling, add read and write interactions, show transaction status through confirmation, and test rejection/revert/wrong-network cases before delivery.

Scope discipline

Do not promise "wallet integration" without asking which chains, wallets, contract addresses, required functions, framework, existing UI, and deployment environment are in scope.

21. 5-Minute Hands-on Challenge

Use the remaining lab time to make the smallest end-to-end dApp path work. The goal is not perfect CSS; it is correct blockchain interaction.

7. Deploy FreelancerBoard to a local chain or test network and record its address.
8. Paste the correct address and ABI into contract.js.
9. Start the React app and connect your practice wallet.
10. Display account, chain ID, contract owner and projectName.
11. Submit setProjectName from the owner account.
12. Show a pending state and transaction hash.
13. Wait for confirmation, then re-read projectName.
14. Switch to a non-owner test account and confirm the contract rejects the privileged write.
15. Switch to the wrong network and confirm your UI blocks the write before submission.

Pass condition	Evidence
Read works	Displayed projectName matches explorer/contract read
Write works	Confirmed transaction + updated state
Authorization works	Non-owner attempt rejected by contract
Network guard works	Wrong chain clearly identified
UX works	No false success before confirmation

Portfolio capture

Take one screenshot of the connected dashboard and one explorer screenshot of the confirmed write. Add architecture + setup + contract address/network to the README.

22. Interview Questions & Quick Quiz

Interview questions

16. What is the difference between a Provider and a Signer?
17. Why does a frontend need an ABI?
18. Why should a dApp wait for a transaction receipt?
19. How do accountChanged and chainChanged affect application state?
20. Why is hiding an admin button not access control?
21. When would you use events in a frontend?
22. What is EIP-1193?
23. What problem does EIP-6963 address?

Quick quiz - answer without notes

24. Which object is appropriate for read-only contract calls?
25. Which object is required for a wallet-authorized state-changing call?
26. What two pieces identify a deployed contract interface for the frontend?
27. What should the UI do after a confirmed write?
28. Name three common transaction failure classes.
29. Why should private keys never be bundled into frontend code?

Answer key

1) Provider-connected Contract. 2) Signer-connected Contract. 3) Contract address + ABI (plus the correct chain context). 4) Re-read authoritative state and update UI. 5) Examples: user rejection, contract revert, insufficient funds, wrong chain, RPC failure. 6) Browser-delivered code is visible to users and cannot safely protect a secret key.

23. Homework - Build a Client-Ready Web3 Dashboard

Required tasks

30. Create a React/Vite project using ethers.js v6.
31. Connect a test wallet through BrowserProvider.
32. Display account and chain ID.
33. Connect to one deployed practice contract using its address + ABI.
34. Display at least three read-only values.
35. Implement at least one state-changing function.
36. Show awaiting-signature, pending, confirmed and error UI states.
37. Handle account and chain changes.
38. Listen for at least one contract event or refresh state after confirmation.
39. Write a README explaining architecture, target network, deployment address, setup commands and security assumptions.

Stretch tasks

- Add an explorer link for the active address and transaction hash.
- Display an ERC-20 balance using decimals correctly.
- Add a switch-network request with a safe fallback explanation.
- Support more than one wallet provider using an EIP-6963-aware integration/library.
- Add component tests by mocking the Web3 service boundary rather than requiring a real wallet in every test.

Freelancer grading rubric

Correct chain/address/ABI (25%) | reads/writes (25%) | transaction UX (20%) | error/network handling (15%) | code organization + README (15%).

24. Day 12 Cheat Sheet

Need	ethers.js v6 pattern
Injected wallet provider	<code>new ethers.BrowserProvider(window.ethereum)</code>
Request account access	<code>provider.send("eth_requestAccounts", [])</code>
Signer	<code>await provider.getSigner()</code>
Account	<code>await signer.getAddress()</code>
Network	<code>await provider.getNetwork()</code>
Read contract	<code>new ethers.Contract(address, abi, provider)</code>
Write contract	<code>new ethers.Contract(address, abi, signer)</code>
Native balance	<code>await provider.getBalance(address)</code>
Format ETH	<code>ethers.formatEther(value)</code>
Parse ETH	<code>ethers.parseEther("0.1")</code>
Wait confirmation	<code>await tx.wait()</code>
Listen event	<code>contract.on("EventName", listener)</code>
Remove listener	<code>contract.off("EventName", listener)</code>

Mental model to memorize

UI -> ethers -> Provider/Signer -> ABI -> Contract -> Transaction/Logs -> UI refresh

Day 13 Preview

Next: Web3 Backend & Automation - JSON-RPC from Node.js/Python, event monitoring, server-side reads, transaction services, API design, indexing concepts, databases and safe secret handling.

25. Primary Technical References

Use primary documentation when syntax or wallet behavior changes. The examples in this lecture are aligned with the following sources available at the time of writing (August 2026):

Reference	Location	Use
ethers.js v6 - Getting Started	docs.ethers.org/v6/getting-started/	Provider, Signer and Contract interaction patterns.
ethers.js v6 - Providers	docs.ethers.org/v6/api/providers/	BrowserProvider, JSON-RPC providers and provider APIs.
EIP-1193 - Ethereum Provider JavaScript API	eips.ethereum.org/EIPS/eip-1193	Standard wallet/provider request interface and events.
EIP-6963 - Multi Injected Provider Discovery	eips.ethereum.org/EIPS/eip-6963	Discovery of multiple injected wallet providers.
MetaMask Connect EVM - Ethereum Provider API	docs.metamask.io/metamask-connect/evm/reference/provider-api/	EIP-1193 provider behavior and wallet integration reference.
MetaMask Connect EVM - JSON-RPC API	docs.metamask.io/metamask-connect/evm/reference/json-rpc-api/	Wallet JSON-RPC methods including account/transaction requests.
Solidity ABI Specification	docs.soliditylang.org/en/latest/abi-spec.html	Function/event encoding that frontends consume through ABI definitions.

Professional habit

When upgrading a project from ethers v5 to v6 or changing wallet connection libraries, verify the current official migration/docs instead of blindly copying old snippets from tutorials.

End of Day 12. Completion standard: you can connect a wallet, verify network, read contract state, submit a write, wait for confirmation, react to changes, handle common failures and explain the architecture clearly to a client.